

EXPERIMENTS 21-30

Name: Naresh M

Reg no: 192424381

Course code: DSA0503

Subject : Query Processing for Data Science

EXPERIMENT – 21

Aim

To write a Pandas program to **remove duplicate values from the WHO region column** of the World Alcohol Consumption dataset. This experiment uses the world_alcohol.csv dataset

Procedure

Step 1: Import the Pandas library.

Step 2: Read the world_alcohol.csv dataset.

Step 3: Display the original dataset.

Step 4: Select the WHO region column.

Step 5: Remove duplicate values using drop_duplicates().

Step 6: Display the unique WHO regions.

Code:

```
import pandas as pd

df = pd.read_csv("world_alcohol.csv")

print("Original DataFrame")

print(df)

result = df["WHO Region"].drop_duplicates()

print("\nWHO Regions after removing duplicates")

print(result)
```

Output:

```
WHO Regions after removing duplicates
0      Western Pacific
1      Americas
2      Africa
5      Eastern Mediterranean
6      Europe
Name: WHO Region, dtype: str
```

Result

Thus, the Pandas program to **remove duplicate values from the WHO Region column** of the World Alcohol Consumption dataset was executed successfully, and only the unique WHO regions were displayed.

EXPERIMENT – 22

Aim

To write a Python program using Matplotlib to **draw a line graph** with suitable labels for the **X-axis**, **Y-axis**, and a **title**.

Procedure

Step 1: Import the matplotlib.pyplot library.

Step 2: Create X-axis and Y-axis values.

Step 3: Use plt.plot() to draw the line.

Step 4: Give a suitable label to the X-axis.

Step 5: Give a suitable label to the Y-axis.

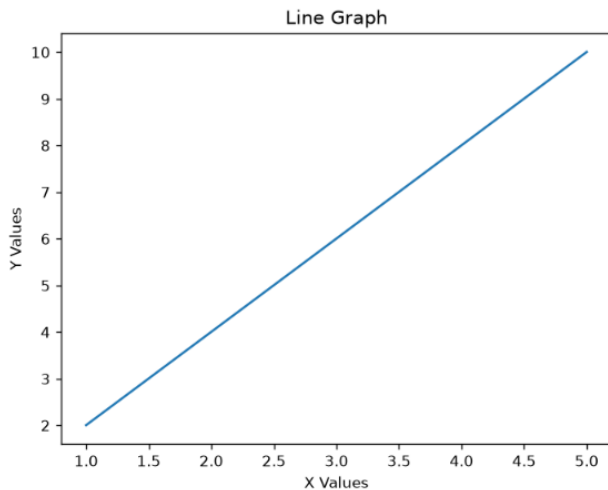
Step 6: Add a suitable title to the graph.

Step 7: Display the graph using plt.show().

Code:

```
import matplotlib.pyplot as plt  
x = [1, 2, 3, 4, 5]  
y = [2, 4, 6, 8, 10]  
plt.plot(x, y)  
plt.xlabel("X Values")  
plt.ylabel("Y Values")  
plt.title("Line Graph")  
plt.show()
```

Output:



Result

Thus, the Python program to draw a line with suitable labels on the X-axis, Y-axis, and a suitable title was executed successfully and the required line graph was displayed.

EXPERIMENT – 23

Aim

To write a Python program to **read X-axis and Y-axis values from a text file** and draw a line graph with suitable labels for the **X-axis, Y-axis, and a title**.

Procedure

- Step 1:** Create a text file named test.txt.
- Step 2:** Store the X-axis and Y-axis values in the text file.
- Step 3:** Import the matplotlib.pyplot library.
- Step 4:** Open and read the test.txt file.
- Step 5:** Separate the values into X and Y coordinates.
- Step 6:** Draw the line using plt.plot().
- Step 7:** Add labels to the X-axis and Y-axis.
- Step 8:** Add a suitable title.
- Step 9:** Display the graph using plt.show().

Code:

```
import matplotlib.pyplot as plt

x = []

y = []

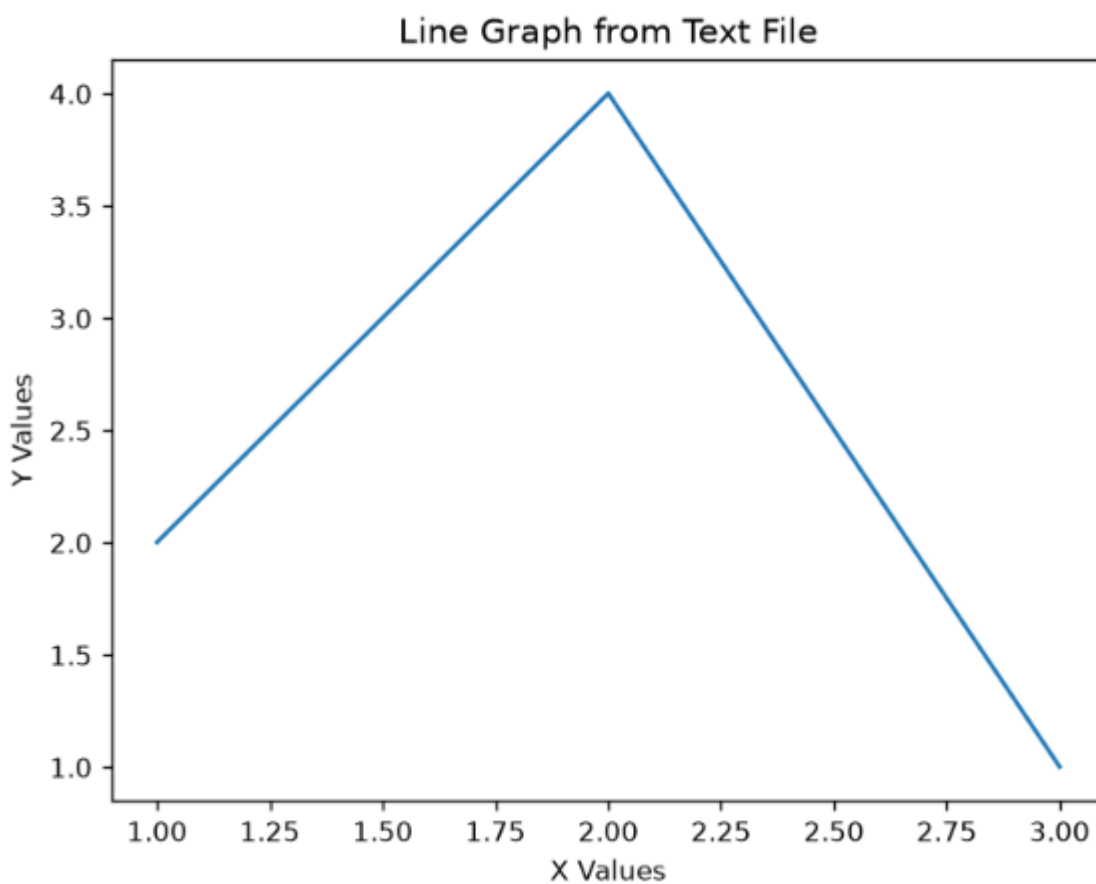
with open("test.txt", "r") as file:

    for line in file:
```

```
values = line.split()
x.append(float(values[0]))
y.append(float(values[1]))

plt.plot(x, y)
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Line Graph from Text File")
plt.show()
```

Output:



Result

Thus, the Python program to read the given axis values from a text file and draw a line graph with suitable X-axis, Y-axis labels and title was executed successfully.

EXPERIMENT – 24

Aim

To write a Python program to draw line charts of the financial data of Alphabet Inc. between October 3, 2016 and October 7, 2016 using the given fdata.csv file.

Procedure

Step 1: Create the fdata.csv file with the given financial data.

Step 2: Import Pandas and Matplotlib.

Step 3: Read the CSV file using pd.read_csv().

Step 4: Convert the Date column into date format.

Step 5: Select the data between **October 3, 2016 and October 7, 2016**.

Step 6: Plot the **Open, High, Low and Close** values.

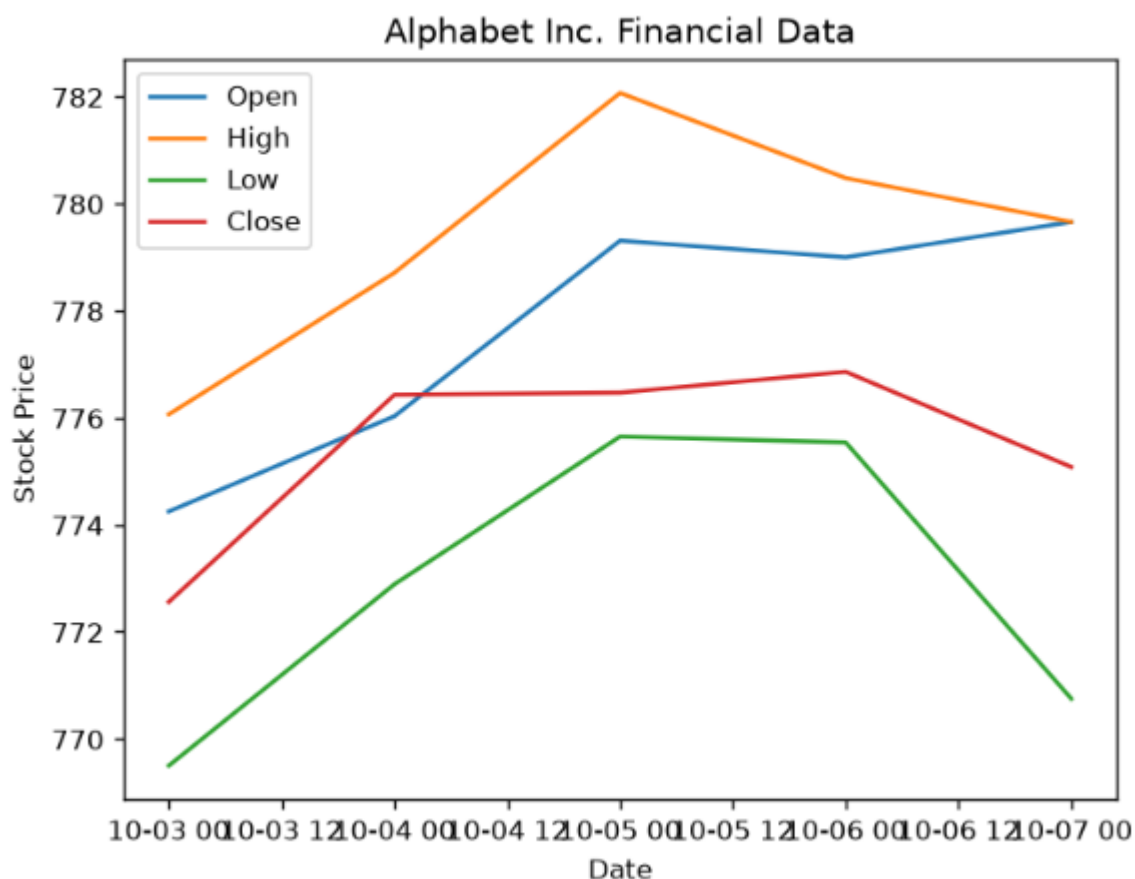
Step 7: Add the X-axis label, Y-axis label, title and legend.

Step 8: Display the line chart.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("fdata.csv")
df["Date"] = pd.to_datetime(df["Date"])
start_date = "2016-10-03"
end_date = "2016-10-07"
data = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]
plt.plot(data["Date"], data["Open"], label="Open")
plt.plot(data["Date"], data["High"], label="High")
plt.plot(data["Date"], data["Low"], label="Low")
plt.plot(data["Date"], data["Close"], label="Close")
plt.xlabel("Date")
plt.ylabel("Stock Price")
plt.title("Alphabet Inc. Financial Data")
plt.legend()
plt.show()
```

Output:



Result

Thus, the Python program to draw line charts of Alphabet Inc.'s financial data between October 3, 2016 and October 7, 2016 was executed successfully and the required line chart was displayed.

EXPERIMENT – 25

Aim

To write a Python program to **plot two or more lines** with suitable **legends, different line widths, and different colors**.

Procedure

Step 1: Import the matplotlib.pyplot library.

Step 2: Define the X-axis values.

Step 3: Define values for two or more lines.

Step 4: Use plt.plot() to draw the first line with a specified color and width.

Step 5: Use plt.plot() again to draw the second line with a different color and width.

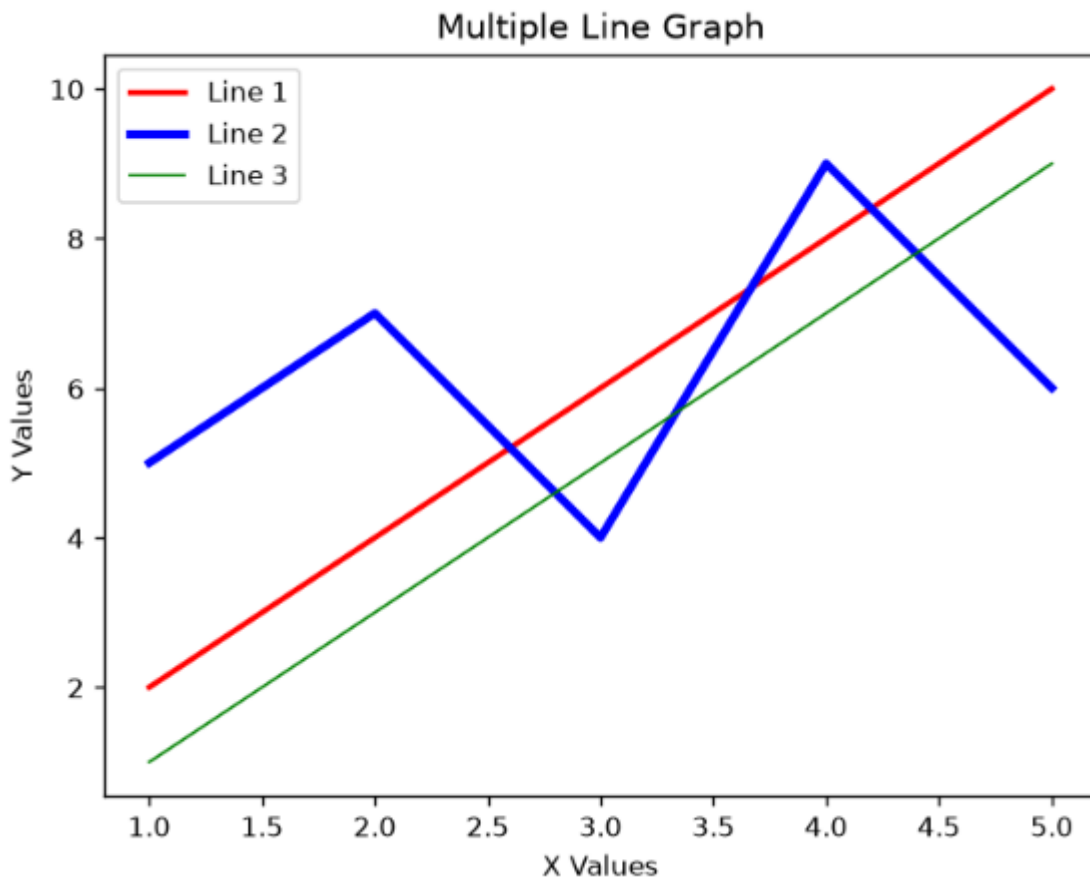
Step 6: Add legends for the lines.

Step 7: Add X-axis label, Y-axis label and title.

Step 8: Display the graph using plt.show().

Code:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y1 = [2, 4, 6, 8, 10]
y2 = [5, 7, 4, 9, 6]
y3 = [1, 3, 5, 7, 9]
plt.plot(x, y1, color="red", linewidth=2, label="Line 1")
plt.plot(x, y2, color="blue", linewidth=3, label="Line 2")
plt.plot(x, y3, color="green", linewidth=1, label="Line 3")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Multiple Line Graph")
plt.legend()
plt.show()
```

Output:

Result

Thus, the Python program to plot two or more lines with legends, different widths and colors was executed successfully and the required multiple-line graph was displayed.

EXPERIMENT – 26

Aim

To write a Python program to create multiple plots using Matplotlib.

Procedure

Step 1: Import the matplotlib.pyplot library.

Step 2: Create the X-axis and Y-axis data.

Step 3: Create multiple plots using plt.subplot().

Step 4: Create a line plot in the first plot.

Step 5: Create a bar plot in the second plot.

Step 6: Create a scatter plot in the third plot.

Step 7: Give suitable titles to the plots.

Step 8: Display all plots using plt.show().

Code:

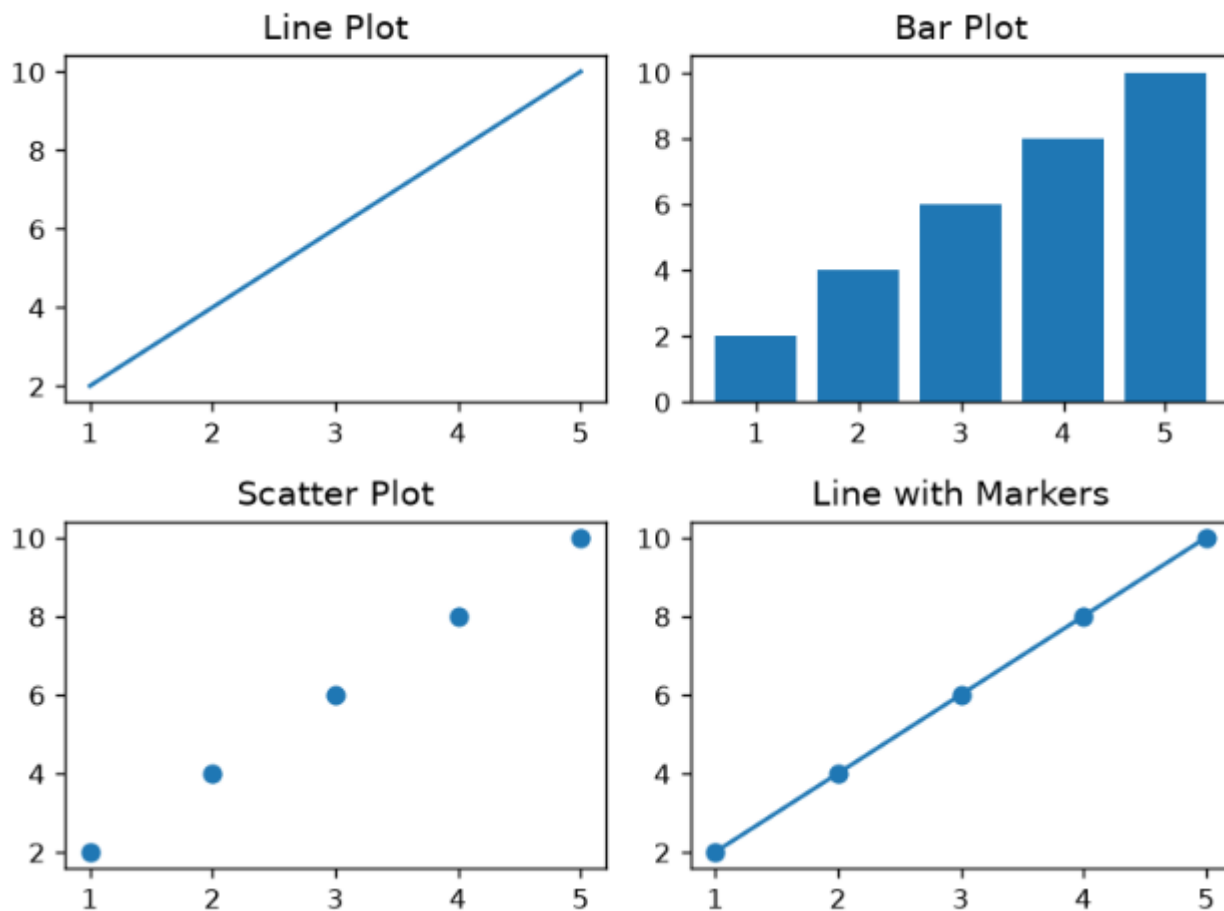
```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.subplot(2, 2, 1)
plt.plot(x, y)
plt.title("Line Plot")
plt.subplot(2, 2, 2)
plt.bar(x, y)
plt.title("Bar Plot")
plt.subplot(2, 2, 3)
plt.scatter(x, y)
plt.title("Scatter Plot")
plt.subplot(2, 2, 4)
plt.plot(x, y, marker="o")
plt.title("Line with Markers")
```



```
plt.tight_layout()
```

```
plt.show()
```

Output:



Result

Thus, the Python program to create multiple plots was executed successfully and the required multiple plots were displayed.

EXPERIMENT – 27

Aim

To write a Python program to display a bar chart representing the popularity of different programming languages.

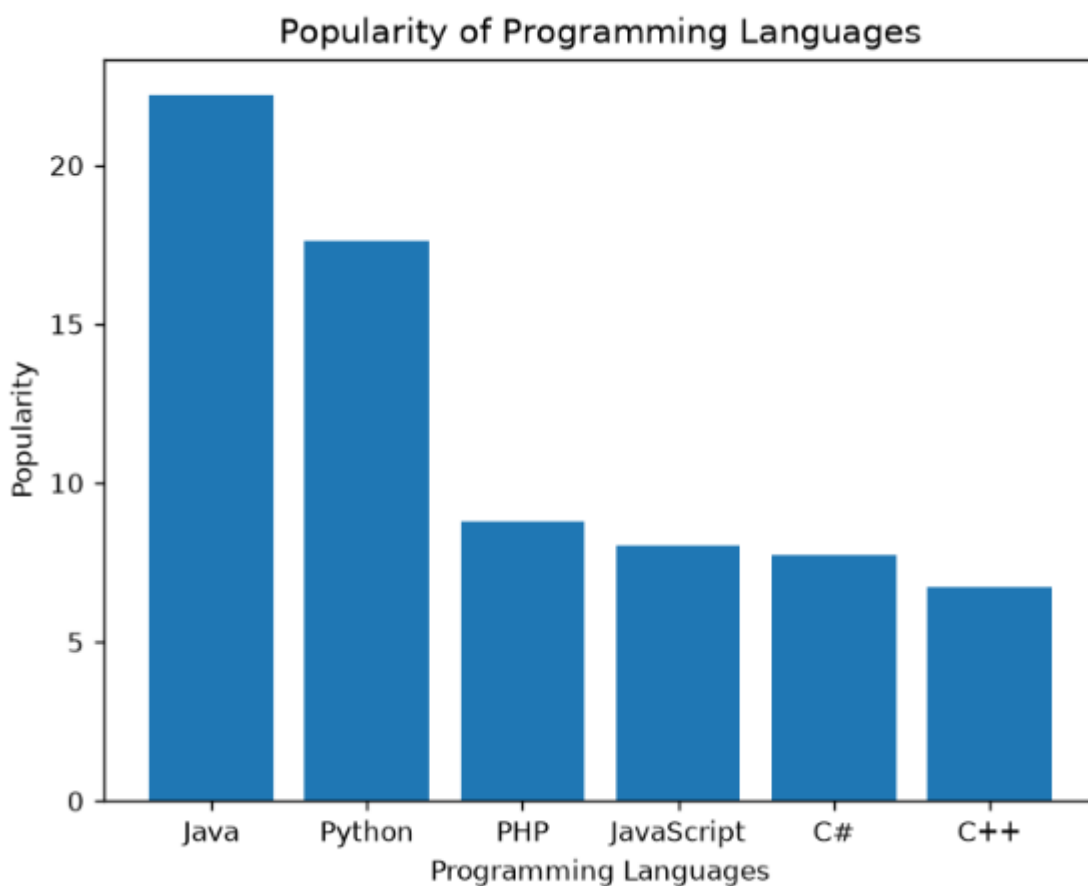
Procedure

1. Import the Matplotlib library.
2. Store the programming languages in a list.
3. Store their popularity values in another list.
4. Create a bar chart using `plt.bar()`.
5. Add X-axis and Y-axis labels.

6. Add a suitable title.
7. Display the graph.

Code:

```
import matplotlib.pyplot as plt
languages = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]
popularity = [22.2, 17.6, 8.8, 8, 7.7, 6.7]
plt.bar(languages, popularity)
plt.xlabel("Programming Languages")
plt.ylabel("Popularity")
plt.title("Popularity of Programming Languages")
plt.show()
```

Output:**Result**

Thus, the Python program to display a bar chart of the popularity of programming languages was executed successfully.

EXPERIMENT – 28

Aim

To write a Python program to display a horizontal bar chart representing the popularity of different programming languages.

Procedure

1. Import the matplotlib.pyplot library.
2. Create a list containing the programming languages.
3. Create a list containing their popularity values.
4. Use plt.barh() to create a **horizontal bar chart**.
5. Add labels for the X-axis and Y-axis.
6. Add a suitable title.
7. Display the chart using plt.show().

Code:

```
import matplotlib.pyplot as plt

languages = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]

popularity = [22.2, 17.6, 8.8, 8, 7.7, 6.7]

plt.barh(languages, popularity)

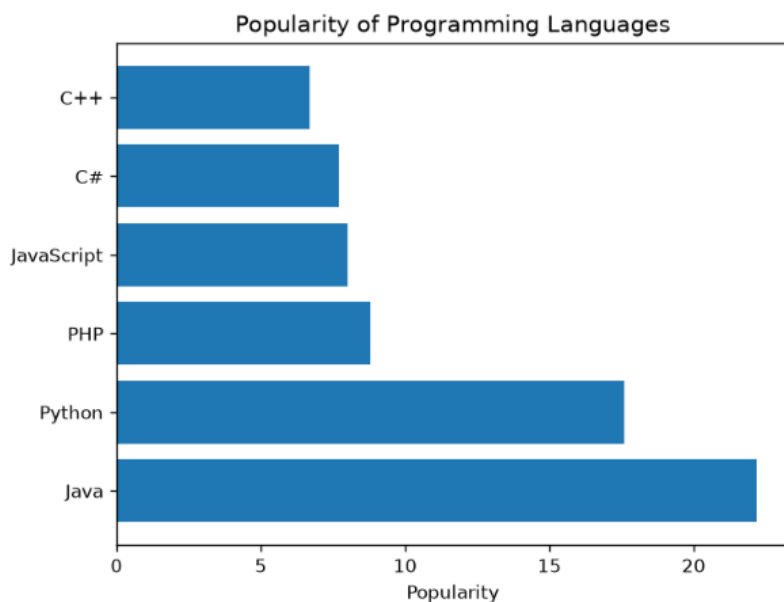
plt.xlabel("Popularity")

plt.ylabel("Programming Languages")

plt.title("Popularity of Programming Languages")

plt.show()
```

Output:



Result

Thus, the Python program to display a horizontal bar chart of the popularity of programming languages was executed successfully and the required horizontal bar chart was displayed.

EXPERIMENT – 29

Aim

To write a Python program to display a bar chart of the popularity of programming languages, using a different color for each bar.

Procedure

1. Import the matplotlib.pyplot library.
2. Create a list of programming languages.
3. Create a list of popularity values.
4. Create a list containing a different color for each bar.
5. Use plt.bar() to create the bar chart.
6. Add X-axis and Y-axis labels.
7. Add a suitable title.
8. Display the chart using plt.show().

Code:

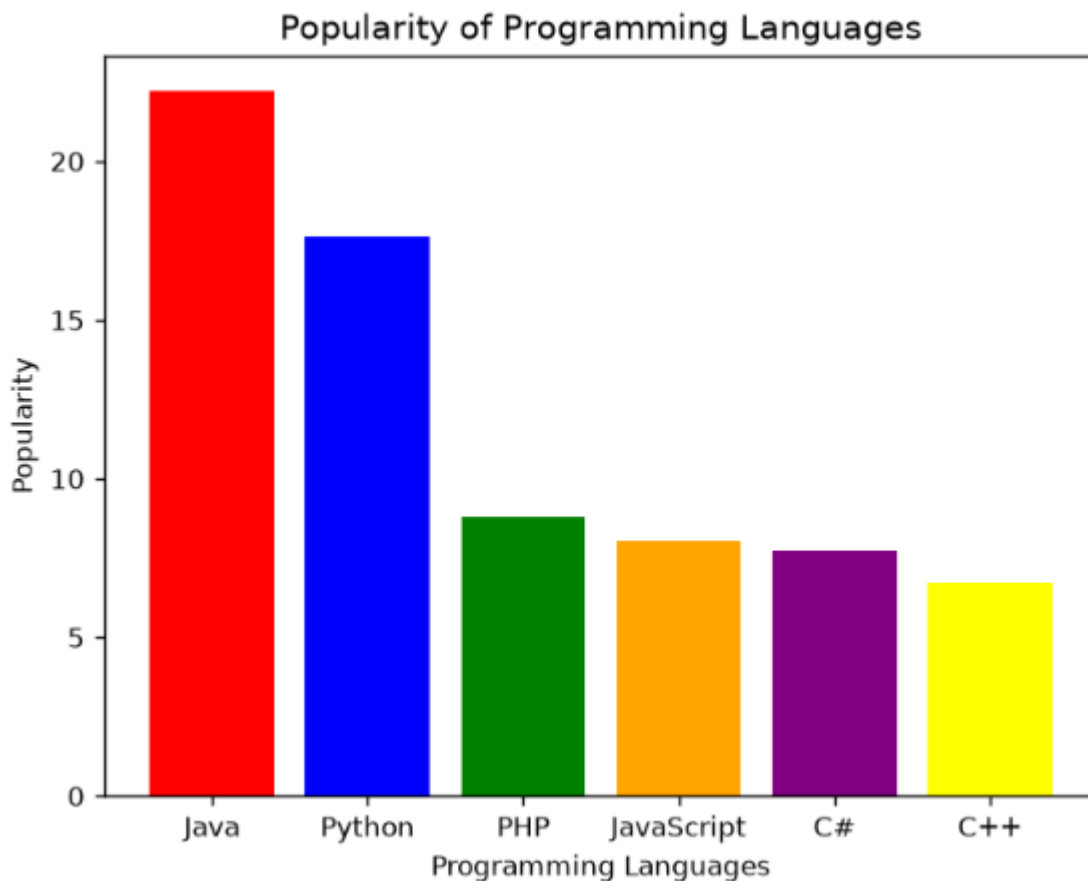
```
import matplotlib.pyplot as plt

languages = ["Java", "Python", "PHP", "JavaScript", "C#", "C++"]
popularity = [22.2, 17.6, 8.8, 8, 7.7, 6.7]
colors = ["red", "blue", "green", "orange", "purple", "yellow"]

plt.bar(languages, popularity, color=colors)

plt.xlabel("Programming Languages")
plt.ylabel("Popularity")
plt.title("Popularity of Programming Languages")
plt.show()
```

Output:



Result

Thus, the Python program to display a bar chart of programming-language popularity with a different color for each bar was executed successfully.

EXPERIMENT – 30

Aim

To write a Python program to create a bar plot of scores by group and gender, using multiple X values on the same chart for men and women.

Procedure

1. Import matplotlib.pyplot.
2. Define the X-axis group values.
3. Define the mean scores for men.
4. Define the mean scores for women.
5. Create X positions using range().
6. Draw the men's bars.
7. Draw the women's bars beside the men's bars.
8. Add X-axis and Y-axis labels.

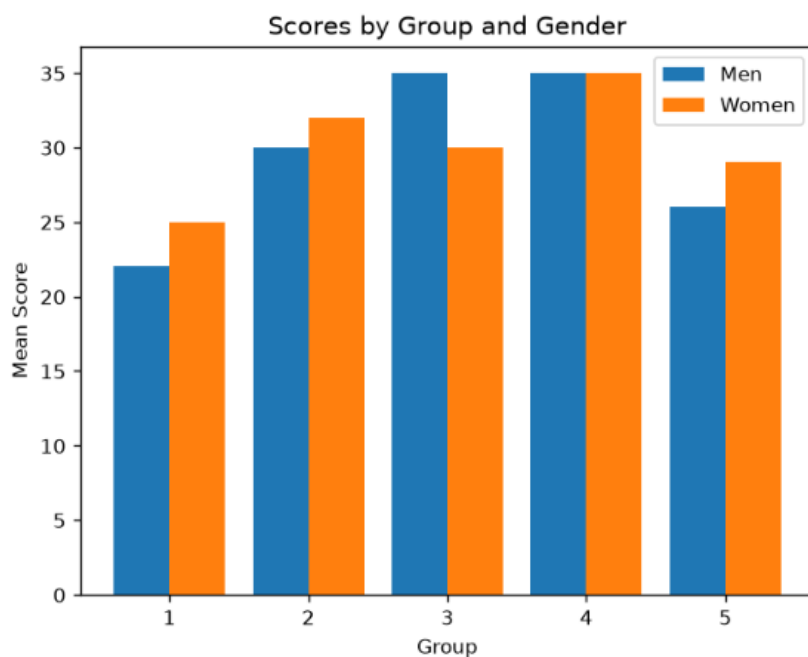
9. Add a title and legend.

10. Display the graph.

Code:

```
import matplotlib.pyplot as plt
men = [22, 30, 35, 35, 26]
women = [25, 32, 30, 35, 29]
groups = [1, 2, 3, 4, 5]
x1 = [i - 0.2 for i in groups]
x2 = [i + 0.2 for i in groups]
plt.bar(x1, men, width=0.4, label="Men")
plt.bar(x2, women, width=0.4, label="Women")
plt.xlabel("Group")
plt.ylabel("Mean Score")
plt.title("Scores by Group and Gender")
plt.xticks(groups)
plt.legend()
plt.show()
```

Output:



Result:

Thus, the Python program to create a bar plot of scores by group and gender using multiple X values for men and women was executed successfully and the required grouped bar chart was displayed.